DSA0310 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1

Morphology-Aware Text Normalisation and Search for a Hospital Record System

Student Details	
Name	V.Lakshmi Prasad
Reg Number	192425179
Department	AI&ML
Subject	DSA0310 – Natural Language Processing
Submission Date	

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

1.1 Levels of linguistic analysis

Level	Meaning	Hospital-search example	Need
Phonology	Sound patterns and pronunciation.	Different pronunciations of a drug name may cause speech-recognition errors.	Not required for typed text; useful only if voice input is added.
Morphology	Internal structure of words and grammatical forms.	fracture, fractures, fractured, fracturing.	Essential.
Syntax	How words combine into phrases and sentences.	"No evidence of fracture" differs from "evidence of fracture".	Useful for phrase context and negation; not the main focus of the finite-state layer.
Semantics	Meaning of words and sentences.	"renal" is related in meaning to "kidney".	Important for clinically meaningful expansion; requires terminology/knowledge resources.
Pragmatics	Meaning based on speaker intention and situation.	"Rule out pneumonia" is not a confirmed diagnosis.	Needed for advanced clinical interpretation, not basic token normalisation.
Discourse	Meaning across multiple sentences or sections.	A later note may say that a suspected condition was ruled out.	Useful for longitudinal records; outside the basic normalisation layer.

The core system needs tokenisation, entity recognition and morphology. Syntax, semantics, pragmatics and discourse become necessary when the hospital wants to distinguish negation, suspected diagnoses, historical conditions and relations across notes.

1.2 Model families

Model family	Best hospital task	Why it fits	Why the other families are less suitable
State machines (finite automata/transducers)	Validate patient IDs, dates and analyse word forms.	They process symbols through explicit states and are efficient for regular patterns.	A probabilistic model is unnecessary for a fixed ID format; a general rule system alone does not naturally represent state transitions.
Formal rule systems	Apply spelling rules, abbreviation expansion and morphological changes.	Rules are transparent, testable and easy to audit.	A state machine alone does not express all contextual clinical knowledge; probability is not needed for deterministic spelling rules.
Probabilistic models	Rank ambiguous OCR corrections or choose likely language/context.	They can use context and uncertainty when several interpretations are possible.	A deterministic regex is too rigid for noisy uncertain input; hand-written rules become very large for all contextual variations.

1.3 Ambiguity in clinical text

- Lexical ambiguity: “lead” may refer to a metal, a cardiac lead or the verb “lead”.
- Morphological ambiguity: “reading” may be a noun or a verb form; “findings” may be a plural noun or part of a fixed expression.

- Segmentation ambiguity: “10mg” may be tokenised as one token or as “10” + “mg”; the design treats it as one measurement token.
- Abbreviation ambiguity: “MS” may mean multiple sclerosis, mitral stenosis or morphine sulfate depending on context.

1.4 Search and resources

Automata perform state-space traversal. A DFA processes each character once, normally in O(n) time and O(1) working memory when only the current state is stored. An NFA may track a set of active states; subset construction can increase the number of states up to 2^n. Regex engines may use backtracking, which can consume additional time and stack/memory, especially with ambiguous nested quantifiers. Lexicon construction, annotation and relevance judgements consume expert effort and labelled data.

1.5 Proposed pipeline

Clinical records

|

v

[Input cleaning] -- Unicode cleanup / OCR noise flags

| Formalism: deterministic rules

v

[Tokenisation] -- words, abbreviations, numbers + units

| Formalism: regular expressions / maximal munch

v

[Entity recognition] -- IDs, dates, BP, drug strengths, etc.

| Formalism: regex + DFA validators

v

[Normalisation] -- case, dates, units, abbreviations

| Formalism: rewrite rules

v

[Morphological analysis] -- stem + POS + features

| Formalism: lexicon FST + orthographic rules

v

[Search index] -- canonical terms and entity fields

| Formalism: inverted index

v

Search results + highlighted source spans

Input: free-text discharge summaries, prescriptions, operation notes and nursing observations. Output: searchable canonical tokens, structured entities and morphological analyses while preserving the original note for audit.

TASK 2 – Basic Regular Expression Patterns

2.1 Basic operators

Operator	Meaning	Clinical example
Concatenation	Items occur in sequence.	AB-12345
Disjunction |	Either alternative.	mg|mcg
Kleene star *	Zero or more repetitions.	[0-9]*
Plus +	One or more repetitions.	[0-9]+
Optional ?	Zero or one occurrence.	-? for optional minus sign
Character class []	One character from a set.	[A-Z]
Range	Continuous character range.	[0-9]
Negated class [^]	Any character except listed ones.	[^\s]+
Anchors ^ $	Beginning and end of string.	^AB[0-9]{5}$
Counters {m,n}	Bounded repetition.	[0-9]{2,4}

2.2 Twelve entity patterns

Entity	Illustrative Python regex	Match / non-match rule
Patient identifier	\\bPT-[0-9]{6}\\b	Matches PT- followed by exactly six digits. Does not accept letters, missing digits or extra digits.
Admission date	\\b(?:[0-3][0-9])[-/](?:[01][0-9])[-/](?:20[0-9]{2})\\b	Matches DD-MM-YYYY or DD/MM/YYYY shape. Calendar validity must be checked separately.
Discharge date	\\b(?:[0-3][0-9])[-/](?:[01][0-9])[-/](?:20[0-9]{2})\\b	Same date format; the field/section

		context distinguishes discharge date from admission date.
Drug name + strength	\\b(?:paracetamol|amoxicillin|metformin|ibuprofen)\\s+[0-9]+(?:\\.[0-9]+)?\\s?(?:mg|mcg|g|ml)\\b	Matches listed drug names with numeric strength and unit. Does not identify unlisted drugs or prove a clinically valid dose.
Dosage frequency	\\b(?:OD|BD|TDS|QID|once daily|twice daily|three times daily)\\b	Matches common frequency expressions. Does not infer frequency from arbitrary prose.
Laboratory value + unit	\\b[A-Za-z][A-Za-z0-9-]*\\s*[:=]?\\s*[0-9]+(?:\\.[0-9]+)?\\s?(?:mg/dL|g/dL|mmol/L|IU/L|%|10\\^?[0-9]+/L)\\b	Matches a test label followed by value and supported unit.
Blood pressure	\\b(?:[0-9]{2,3})\\s*/\\s(?:[0-9]{2,3})\\s?mmHg\\b	Matches systolic/diastolic numeric form with mmHg. Range checking is separate.
Temperature	\\b(?:[3-4][0-9](?:\\.[0-9])?)\\s?°?C\\b	Matches a Celsius temperature in a broad clinical range.
ICD code	\\b[A-Z][0-9]{2}(?:\\.[A-Z0-9]{1,4})?\\b	Matches a simplified ICD-like code shape; official code-set validation is required.
Ward and bed	\\bWard\\s+[A-Z0-9-]+\\s*,?\\s*Bed\\s+[A-Z0-9-]+\\b	Matches a ward and bed label.

Contact number	\\b(?:\\+91[-]?)?[6-9][0-9]{9}\\b	Matches a common Indian mobile number shape, optionally with +91.
Age	\\b(?:[0-9]|[1-9][0-9]|1[01][0-9]|120)\\s?(?:years?|yrs?|y/o)\\b	Matches age from 0 to 120 in supported formats.

Duration of symptom is included as a thirteenth useful pattern because it appears in the assignment inventory discussion: r'\b[0-9]+\s?(?:hours?|days?|weeks?|months?)\b'. It matches “3 days” but not vague expressions such as “for a long time”.

The date regex checks the shape only. For example, 31/02/2026 matches the shape but is not a valid calendar date. A date parser must perform the semantic calendar check.

2.3 Greedy and non-greedy matching

Sentence: “Give 500 mg paracetamol after food; repeat 250 mg if prescribed.”\nGreedy: r"Give .* mg" -> may span from “Give” to the last “mg”.\nNon-greedy:r"Give .*? mg" -> stops at the first “mg”.

The greedy quantifier .* consumes as much text as possible while still allowing the rest of the pattern to match. The non-greedy .*? consumes as little as possible, so it ends at the earliest valid “mg”. For clinical extraction, a bounded pattern around a drug name and number is safer than a broad .* expression.

2.4 Precedence and grouping

Concatenation binds more tightly than alternation. The expression ab|cd means (ab)|(cd), not a(b|c)d. For example, ^Ward|Bed[0-9]+$ is interpreted as (^Ward) OR (Bed followed by digits at the end), so it may accept the word “Ward” without a bed number. The intended expression is ^(?:Ward|Bed)[0-9]+$.

2.5 Evaluation approach and illustrative error analysis

For every pattern, manually label a test set as correct entity spans or non-entities. TP = correctly matched entities; FP = incorrect matches; FN = missed entities. Precision = TP/(TP+FP); recall = TP/(TP+FN). Typical causes: FP from overly broad formats or ambiguous abbreviations; FN from spelling variants, punctuation, OCR errors and unsupported units.

Pattern group	Illustrative TP	FP	FN	Likely cause
Patient ID / contact	48	1	2	Extra separator and unanticipated country prefix.
Dates	35	3	4	Impossible calendar date accepted; two-digit year missed.
Drug + strength	42	4	6	Unlisted drug and OCR confusion between 0/O.
Vitals / lab values	51	5	5	Unusual unit and missing unit spacing.
Other entities	39	3	7	ICD version differences and local ward abbreviations.

These counts are examples only; they must be replaced by counts from the actual annotated corpus.

TASK 3 – Regular Expressions for Tokenisation and Normalisation

3.1 Token definition and tokenizer

A token is a meaningful searchable unit. Keep decimals, common abbreviations with internal full stops, hyphenated clinical terms and number-unit combinations together. Separate ordinary punctuation. A practical Python tokenizer is:

```
TOKEN_RE = re.compile(
    r"""(?ix)
    (?:[A-Z]\.){2,}                    # e.g. U.S.
    | [A-Za-z]+(?:[-'][A-Za-z]+)*\.?   # words, hyphens, apostrophes
    | [0-9]+(?:\.[0-9]+)?(?:mg|mcg|ml|g|kg|mmHg|°C)? # values + units
    | [0-9]{1,2}[-/][0-9]{1,2}[-/][0-9]{2,4} # dates
    | [^\\w\\s]                        # punctuation
    """
)
```

3.2 Maximal-munch examples

Input	Whitespace split	Maximal-munch tokens
“BP 120/80 mmHg”	BP | 120/80 | mmHg	BP | 120/80 | mmHg
“take 500mg BD”	take | 500mg | BD	take | 500mg | BD
“post-operative review”	post-operative | review	post-operative | review
“e.g. renal function”	e.g. | renal | function	e.g. | renal | function

Maximal munch selects the longest valid token at the current position. It prevents values such as 12.5mg from being split into unrelated fragments. The tokenizer should preserve character offsets so search results can highlight the original text.

3.3 Normalisation rules and abbreviation list

Apply case folding, whitespace cleanup, date canonicalisation and unit normalisation. Abbreviations must be expanded only when context makes the meaning sufficiently certain.

Abbreviation	Expansion / canonical form
BP	blood pressure
HR	heart rate

RR	respiratory rate
Temp	temperature
SOB	shortness of breath
N/V	nausea and vomiting
c/o	complains of
h/o	history of
DM	diabetes mellitus
HTN	hypertension
CKD	chronic kidney disease
AKI	acute kidney injury
MI	myocardial infarction (context check)
CVA	cerebrovascular accident
UTI	urinary tract infection
OD	once daily (context check)
BD	twice daily
TDS	three times daily
QID	four times daily
PO	by mouth
IV	intravenous
IM	intramuscular
SOS	if required
PRN	as needed
NPO	nothing by mouth
O/E	on examination
WNL	within normal limits
CBC	complete blood count
LFT	liver function test
RFT	renal function test
ECG	electrocardiogram

Example synthetic record before normalisation: “PT-123456 admitted 01/11/2026. c/o fever x 3 days. BP 120/80 mmHg; Temp 38.2C. Rx: Paracetamol 500mg BD.”

After normalisation: “pt-123456 admitted 2026-11-01. complains of fever for 3 days. blood pressure 120/80 mmHg; temperature 38.2 °C. prescription: paracetamol 500 mg twice daily.”

3.4 Irreversible normalisation decisions

- Case folding removes original capitalisation. Usually acceptable for ordinary search, but not for case-sensitive identifiers or gene symbols.
- Abbreviation expansion replaces a short form with a longer phrase. It can destroy the original wording and may choose the wrong sense; retain the original token and store the expansion as an additional indexed field.
- Unit conversion changes the displayed value and may introduce rounding. Keep the original value/unit alongside the canonical value.
- Removing punctuation can erase distinctions in codes and decimals. Preserve the raw source string and offsets.

3.5 A requirement beyond regular expressions

A classical regular expression/finite automaton cannot recognise arbitrarily nested balanced parentheses, because that language requires unbounded memory (a stack). A clinical note containing nested explanatory brackets therefore needs a stack-based parser or a more powerful grammar formalism. Practical regex engines sometimes add non-regular extensions, but those are not classical finite automata.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence

4.1 Formal definition

A finite automaton is a five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 Four DFA validators (compact transition descriptions)

To keep the tables readable, transitions are grouped by character class. Any transition not listed goes to a dead state $\perp$. The dead state loops to itself.

Validator / states	Transition sequence	Accepting condition
Patient ID: q0…q9	q0 --P--> q1 --T--> q2 --−--> q3; six digit transitions q3→q9.	q9 only; exact PT- plus six digits.
Mobile: q0…q10	q0 --[6-9]--> q1; each of next nine digits advances one state.	q10 only; ten digits.
BP: q0…q8	2–3 digit systolic field; slash; 2–3 digit diastolic field; optional space then mmHg.	Final unit-complete state.
ICD-like code: q0…q6	Uppercase letter; two digits; optional dot; one to four alphanumeric characters.	After two digits, or after valid dotted suffix.

Example patient-ID transition table:

Current state	Input	Next state
q0	P	q1
q1	T	q2
q2	-	q3
q3	digit	q4
q4	digit	q5
q5	digit	q6
q6	digit	q7
q7	digit	q8
q8	digit	q9

q9	end of input	ACCEPT

Diagram (patient identifier): q0 —P→ q1 —T→ q2 —-→ q3 —digit→ q4 —digit→ q5 —digit→ q6 —digit→ q7 —digit→ q8 —digit→ q9(accept).

4.3 NFA with optional country prefix

Consider a contact-number pattern with an optional +91 prefix: (+91)? followed by a ten-digit mobile number. The NFA can branch at the start: either skip the prefix using ε or consume +91.

```
q0 --ε--> q1 --[6-9]--> q2 --digit--> ... --digit--> q10 (accept)
 |
 +-- '+' --> q11 --'9'--> q12 --'1'--> q1
```

Accepted trace for 9876543210: ε-closure(q0) = {q0,q1}; then 9 moves to q2; each following digit advances one state; after ten digits the accepting state q10 is active. Rejected example 5876543210: the first digit 5 has no transition from q1, so the active-state set becomes empty.

4.4 Subset construction and minimisation

For the optional prefix NFA, subset construction creates DFA states from sets of NFA states. Start state is ε-closure({q0}). For each input symbol, take all NFA transitions and then ε-close the result. The empty set is the dead state. Because the NFA branches only at the optional prefix, the DFA can be minimised by merging equivalent states only when they have identical future acceptance behaviour.

DFA subset state	Input class	Next subset	Meaning
A={q0,q1}	+	B={q11}	Prefix chosen
A={q0,q1}	[6-9]	C={q2}	No prefix; first mobile digit
B={q11}	9	D={q12}	Prefix second character
D={q12}	1	C={q1}	Prefix completed
C={q1}	[6-9]	E={q2}	Mobile number begins
E={q2}	digit	F={q3}	Continue digits
F…	digit	next position	Continue until ten digits
Any state	invalid symbol	∅	Dead state

A production submission should list every position-state for all ten digits and calculate the minimised state count from the implemented transition table. The compact table above shows the construction logic; the exact count depends on whether prefix and no-prefix paths are merged after they reach equivalent suffix states.

4.5 Thompson construction and equivalence

For the simple regex [0-9]+, Thompson construction creates a fragment for one digit and connects it to a loop for zero or more additional digits. The automaton accepts exactly one or more digits. Conversely, eliminating states from a digit-loop automaton yields the regular expression [0-9]+. Thus each transition path corresponds to a string described by the regex, and each string described by the regex has a path to an accepting state.

4.6 Pumping lemma proof

The pumping lemma states that for every regular language L there is a pumping length p such that every string s in L with $|s| \geq p$ can be written s = xyz, where $|xy| \leq p$, $|y| > 0$, and xy^iz belongs to L for every $i \geq 0$.

Let $L = \{ '('^n ')'^n : n \geq 0 \}$, the language of strings with equal numbers of opening and closing parentheses in the required order. Assume L is regular and choose $s = '('^p ')'^p$. Since $|xy| \leq p$, the non-empty y consists only of opening parentheses. Pumping y down (i=0) leaves fewer opening than closing parentheses, so the result is not in L. This contradicts the lemma. Therefore L is not regular.

TASK 5 – Inflectional Morphology

5.1 Key terms and distinction

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the base to which an affix attaches. An affix is a bound morpheme added before or after a stem. Inflection changes grammatical features without normally changing word class or core meaning. A paradigm is the set of forms belonging to one lexeme.

Inflection is generally category-preserving and highly productive (fracture → fractures). Derivation can change word class or meaning (operate → operation), is less predictable and may occur in different affix orders.

5.2 Clinical inflectional paradigms

Lexeme	Noun number	Verb forms	Notes
fracture	fracture / fractures	fracture / fractures / fractured / fracturing	-s, -ed, -ing
infect	infection / infections	infect / infects / infected / infecting	Verb and derived noun family
treat	treatment / treatments	treat / treats / treated / treating	-s, -ed, -ing
observe	observation / observations	observe / observes / observed / observing	Silent-e spelling change
bleed	bleedings (rare) / bleeding	bleed / bleeds / bled / bleeding	Irregular past: bled

Subject agreement in English includes forms such as the patient recovers versus the patients recover. Adjective comparison may occur in general clinical prose: mild, milder, mildest; severe, more severe, most severe.

5.3 Orthographic rules

Rule	Implementation description	Counter-example / caution
e-deletion	Drop final silent e before -ing: observe + ing → observing.	See + ing → seeing; e is not deleted from “ee”.
Consonant doubling	Double final consonant after a	Visit + ed → visited; do not

	short stressed vowel in suitable monosyllabic forms: admit + ed → admitted.	double t.
y-to-i	Consonant + y changes y to i before -es/-ed: biopsy → biopsies; carry → carried.	Vowel + y: play → played, not plaied.
sibilant -es	Add -es after s, x, z, ch, sh: match → matches.	Photo → photos; do not add -es to every final o.

5.4 Irregular forms

Base	Irregular form	Why lexicon listing is needed
be	am, is, are, was, were, been	Suppletion and multiple forms cannot be generated by one productive suffix rule.
go	went, gone	Past form is suppletive.
have	has, had	Stem alternation and irregular past.
do	does, did, done	Irregular forms.
bleed	bled	Vowel change and suffix pattern differ from regular -ed.
write	wrote, written	Stem vowel and participle change.
diagnosis	diagnoses	Special spelling/number pattern.
analysis	analyses	Greek-derived plural.

5.5 Measuring inflection effect

Type-to-lemma ratio = number of distinct surface word types / number of distinct lemmas. A lower ratio after lemmatisation indicates that several surface forms have been grouped under one lemma. Query failures explained by inflection = failures caused only by a mismatch in inflected form / total query failures × 100.

Illustrative calculation: if a synthetic corpus has 8,400 surface types and 6,000 lemmas, the ratio is 1.40. If 18 of 60 failed query cases are caused only by inflection, inflection explains 30% of failures. These are demonstration numbers, not measured corpus results.

TASK 6 – Derivational Morphology

6.1 Derivational affix inventory

Affix	Example	Typical effect
-itis	arthritis	Noun: inflammation condition
-ectomy	nephrectomy	Noun: surgical removal
-ology	neurology	Noun: field of study
-osis	fibrosis	Noun: condition/process
-aemia	anaemia	Noun: blood condition
-ation	ventilation	Verb → noun
-ment	treatment	Verb → noun
-able	treatable	Verb → adjective
-ive	responsive	Verb/noun base → adjective
-al	renal	Noun → adjective
-ic	cardiac	Noun → adjective
nephro-	nephrology	Prefix: kidney-related
cardio-	cardiology	Prefix: heart-related
hyper-	hypertension	Prefix: excessive/high
hypo-	hypoglycaemia	Prefix: low/below
re-	reoperation	Prefix: again
un-	unresponsive	Prefix: not/opposite

6.2 Derivational families and order

Root	Family / attachment order	Impossible or unsuitable order
nephr (kidney)	nephr + -ectomy → nephrectomy; nephr + -ology → nephrology	-ectomy + nephro- is not a normal English attachment order.
cardi (heart)	cardi + -ac → cardiac; cardio- + -logy → cardiology	cardiac + -ology is not the normal route to cardiology.
infect	infect + -ion → infection; infection + -s → infections	-s before -ion (*infectsion) is invalid.

treat	treat + -ment → treatment; treat + -able → treatable	-ment + -able (*treatmentable) is generally not intended.
operate	operate + -ion → operation; re- + operation → reoperation	operation + re- (*operationre) is invalid.
respond	respond + -ive → responsive; un- + responsive → unresponsive	responsive + un- (*responsiveun) is invalid.

6.3 Semantic non-compositionality and search

A derived clinical term may have a specialised meaning that cannot be calculated simply by combining its parts. “Nephrectomy” means surgical removal of a kidney, not merely a generic kidney-related event. “Anaemia” is a defined clinical condition, not any state involving blood. Aggressive stemming may merge unrelated words or remove meaningful distinctions. A clinical search system should use terminology-aware mappings rather than strip every suffix.

6.4 Productivity and conflation decision

Productivity can be estimated by counting distinct derived types and total tokens containing each affix. An affix with many different new bases is more productive; a fossilised affix appears in a small, fixed set of words. Without the actual corpus, exact counts cannot be claimed. A practical initial classification is: -ation, -able, re-, un- (productive); -ectomy, -itis, -aemia, -osis (more specialised and often lexicon-driven).

- Helpful conflation: fracture ↔ fractures/fractured; infection ↔ infections.
- Helpful but controlled: nephrectomy ↔ nephrectomies; the plural should retrieve the singular procedure.
- Misleading conflation: renal ↔ nephrectomy; related domain but not equivalent concept.
- Misleading conflation: hypertension ↔ hypotension; shared form but opposite clinical meaning.

Recommendation: conflate inflectional variants by default. Conflate derivational relatives only through curated clinical terminology relations and keep the relation type visible.

TASK 7 – Finite-State Morphological Parsing

7.1 Finite-state transducer

A finite-state transducer (FST) is a finite-state machine with two symbol tapes. The lexical tape contains the lemma and grammatical features; the surface tape contains the written word. A transition may consume/emit symbols on the two tapes, including ε where one side has no corresponding symbol.

Example: lexical input fracture +N +PL maps to surface output fractures. In parsing direction, the surface form fractures maps back to fracture +N +PL.

7.2 Lexicon and continuation classes

Class	Example entries	Allowed continuation
NOUN	fracture, infection, kidney, patient	Noun plural -s/-es; selected derivational suffixes
VERB	treat, observe, admit, bleed	3SG -s, past -ed or irregular form, progressive -ing
ADJECTIVE	mild, severe, responsive	comparative/superlative or derivational prefixes where valid
DERIVED_NOUN	treatment, operation, nephrectomy	Plural suffix where licensed

A practical lexicon should contain at least 300 stems, with separate tables for regular stems, irregular forms and clinical terminology. The small examples here demonstrate the structure but do not satisfy the 300-stem resource requirement.

7.3 Hand traces

Surface form	Lexical analysis	Consumed / emitted operation
fractures	fracture +N +PL	fracture stem; surface suffix s ↔ +PL
treated	treat +V +PAST	treat stem; ed ↔ +PAST
observing	observe +V +PROG	final e deleted; ing ↔ +PROG
infections	infection +N +PL	infection stem; s ↔ +PL

unresponsive	un- + responsive +ADJ	un- prefix and adjective stem

7.4 Generation and ambiguity

The same FST can be traversed in reverse: lexical input fracture +N +PL generates fractures. No separate generator is required; reversing the transducer's input/output labels provides generation, subject to the implementation's treatment of ε transitions.

Ambiguous examples: "reading" may be read +V +PROG or reading +N; "treated" may be treat +V +PAST or an adjective in a phrase such as "treated surface". The FST should return all valid analyses. A downstream tagger or sentence-context model selects the most likely analysis, while retaining alternatives when confidence is low.

TASK 8 – Orthographic Rules and Two-Level Morphology

8.1 Why concatenation is insufficient

- observe + ing gives *observeing; correct surface form is observing.
- admit + ed gives *admited; correct surface form is admitted.
- biopsy + es gives *biopsyes; correct surface form is biopsies.
- match + s gives *matchs; correct surface form is matches.

8.2 Rewrite rules

Rule	Rewrite notation	Context / example
Silent-e deletion	e → ε / V C __ +ing	observe + ing → observing (with lexical conditions).
Consonant doubling	C → CC / stressed short V __ +ed	admit + ed → admitted.
Y-to-i	y → i / C __ +es	biopsy + es → biopsies.
Sibilant plural	ε → e / {s,x,z,ch,sh} __ +s	match + s → matches.
Plural -es spelling	y → i / C __ +es	biopsy + es → biopsies.

The rules must be constrained by lexical and phonological conditions; applying them mechanically to every matching string will over-generate. For example, not every final consonant doubles, and vowel+y does not change to i.

8.3 Cascade and two-level approach

Lexicon FST

|

v

Morphological intermediate form

|

v

Rule transducer 1: e-deletion

|

v

Rule transducer 2: consonant doubling

|

v

Rule transducer 3: y-to-i / sibilant plural

|

v

Surface word

A sequential cascade applies ordered rewrite transducers one after another. Its advantage is that each rule is modular and easy to debug; its disadvantage is order dependency and possible interaction between rules. Koskenniemi-style two-level morphology relates lexical and surface strings in parallel using constraints at corresponding positions. It can avoid a long ordered rewrite chain, but rule interactions and implementation can be harder to understand.

8.4 Test plan

Lexical input	Expected surface	Rule	Expected result
observe + V + PROG	observing	e-deletion	Pass
admit + V + PAST	admitted	doubling	Pass
biopsy + N + PL	biopsies	y-to-i	Pass
match + N + PL	matches	sibilant -es	Pass
fracture + N + PL	fractures	regular plural	Pass
treat + V + PAST	treated	regular -ed	Pass
play + V + PAST	played	vowel+y exception	Pass
visit + V + PAST	visited	no doubling	Pass

The full assignment asks for at least 20 test forms. Extend this table with ten additional stems from the built lexicon and record pass/fail against a manually checked expected form. Do not report a success rate until those tests have been run.

TASK 9 – The Porter Stemmer

9.1 Definitions and conditions

The Porter stemmer is a suffix-stripping algorithm for English. It does not require a dictionary and therefore runs quickly, but its output is a stem-like string rather than necessarily a valid lemma.

- Consonant: a letter that is not a, e, i, o or u; y is treated contextually (as a consonant at the start, otherwise opposite to the preceding letter).
- Measure m: the number of vowel-consonant (VC) sequences in the stem pattern.
- *v: the stem contains a vowel.
- *d: the stem ends with a double consonant.
- *o: the stem ends consonant-vowel-consonant where the final consonant is not w, x or y.

9.2 Illustrative measure calculations

Word	Approximate VC pattern	m (illustrative)
fracture	C C V C C V	2
infection	V C C V C V C	3
treating	C C V V C V C	2
admitted	V C C V C V C	3
renal	C V C V C	2
bleeding	C C V V C V C	2

Exact Porter calculations depend on the algorithm's contextual treatment of y and on the stem remaining after earlier steps. The values above are simplified demonstrations; a code implementation should calculate m mechanically.

9.3 Five Porter steps (summary)

1. Step 1: remove plurals and common -ed/-ing endings; apply related adjustments such as -ies → -i and eed handling.
2. Step 2: replace selected longer suffixes when the stem measure is greater than zero, e.g. -ational → -ate.
3. Step 3: remove or replace selected suffixes such as -icate, -ative and -alize under measure conditions.
4. Step 4: remove a list of suffixes only when the remaining stem has m > 1.

5. Step 5: remove terminal e under measure/ending conditions and reduce a final double l in the specified case.

9.4 Worked examples (conceptual)

Word	Likely Porter output	What it illustrates
fractures	fractur	plural -s removal
fractured	fractur	-ed removal
treatments	treatment	plural removal; derivational suffix may remain
infections	infect	plural and -ion-related processing
observing	observ	-ing removal and e restoration conditions
renal	renal	may remain unchanged

For an exact hand trace, execute the reference Porter algorithm and record each intermediate stem after Steps 1a, 1b, 1c, 2, 3, 4 and 5. The output can vary by Porter implementation/version; NLTK's PorterStemmer should be named in the report.

9.5 Stemming versus lemmatisation and retrieval

Stemming strips endings without understanding the lexeme. Lemmatisation uses vocabulary and grammatical features to return a valid base form. For example, "bled" can be lemmatised to "bleed", while a simple suffix stemmer may not recover the correct lemma. "Renal" should remain renal; it should not be forced into a misleading stem.

Method	Precision	Recall	F1	Interpretation
No stemming	0.88	0.62	0.73	Exact forms only; misses inflectional variants.
Porter stemming	0.80	0.76	0.78	Improves recall but can over-stem clinical terms.
Morphological	0.91	0.84	0.87	Best balance in

analysis				this illustrative example; depends on lexicon coverage.

F1 = 2 × precision × recall / (precision + recall). The table is illustrative and must be replaced by measured values from the 40-query relevance set. Over-stemming example: hypertension and hypotension should not be treated as equivalent. Under-stemming example: bled and bleed may fail to meet without an irregular-form lexicon.

TASK 10 – Implement and Test the Pipeline

10.1 Python implementation outline

The following standard-library/NLTK-oriented outline demonstrates the main components. A full submission should place the functions in a .py file, run them on the documented corpus and attach the output logs.

```
import re
from datetime import datetime
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(
    r"(?:[A-Z]\.){2,}|[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"\d+(?:\.\d+)?(?:mg|mcg|ml|g|kg|mmHg)?|"
    r"\d{1,2}[-/]\d{1,2}[-/]\d{2,4}", re.I
)
PATTERNS = {
    "patient_id": re.compile(r"\bPT-\d{6}\b", re.I),
    "mobile": re.compile(r"\b(?:\+91[- ]?)?[6-9]\d{9}\b"),
    "bp": re.compile(r"\b\d{2,3}/\d{2,3}\s?mmHg\b", re.I),
    "temperature": re.compile(r"\b\d{2}(?:\.\d)?\s?°?C\b", re.I),
    "drug_strength": re.compile(
        r"\b(?:paracetamol|amoxicillin|metformin|ibuprofen)\s+"
        r"\d+(?:\.\d+)?\s?(?:mg|mcg|g|ml)\b", re.I)
}
ABBREVIATIONS = {"bp":"blood pressure", "htn":"hypertension",
                 "dm":"diabetes mellitus", "bd":"twice daily",
                 "tds":"three times daily", "prn":"as needed"}

def tokenize(text):
    return TOKEN_RE.findall(text)

def normalise(text):
    text = text.casefold()
```

```
    for short, full in ABBREVIATIONS.items():
        text = re.sub(rf"\b{re.escape(short)}\b", full, text)
    return re.sub(r"\s+", " ", text).strip()

def extract_entities(text):
    return {name: rx.findall(text) for name, rx in PATTERNS.items()}

stemmer = PorterStemmer()
def porter_stem(token):
    return stemmer.stem(token)

# A production FST should be built from a maintained lexicon,
# continuation classes and spelling-rule transducers.
def simple_lexicon_analysis(surface, lexicon):
    if surface in lexicon:
        return lexicon[surface]
    if surface.endswith("s") and surface[:-1] in lexicon:
        return lexicon[surface[:-1]] + " +PL"
    return None
```

10.2 Tests and reporting

Stage	Input count (example only)	Output / metric to record	Rejected / failure count
Tokeniser	25,000 tokens	Tokens produced; runtime	Unrecognised character sequences
Entity regex	All records	Matches per entity type; TP/FP/FN	Invalid format or missed entity
DFA validators	All candidate strings	Accepted / rejected counts	Invalid transition
Morphological analyser	200 annotated forms	Analyses returned; coverage	Out-of-lexicon forms
Porter stemmer	All alphabetic tokens	Stems and runtime	Normally none; quality errors measured

			separately

Test cases must include one valid and one invalid string for every entity pattern, accepted and rejected NFA strings, all ambiguous forms, and irregular forms. Compare implementation outputs with hand calculations from Tasks 4, 7 and 9. Record runtime using time.perf_counter() and repeat runs to reduce noise.

Suggested plots: (1) analysis coverage against lexicon size; (2) runtime against record length. Plot only after collecting actual measurements. A reproducible experiment should record Python/NLTK versions, corpus size, machine details and random seed if sampling is used.

TASK 11 – Compare, Recommend and Defend the Normalisation Design

Strategy	Retrieval quality	Coverage	Runtime	Maintenance
No normalisation	Low recall for variant forms; precise exact matches.	Very high surface coverage but no conflation.	Lowest processing cost.	Low maintenance.
Porter stemming	Often higher recall; precision can fall through over-stemming.	Broad for English words, weak for irregulars and clinical vocabulary.	Very fast.	Low-to-medium; rules are fixed.
Finite-state morphological analysis	Can provide high precision and recall for covered vocabulary.	Limited by lexicon and rule coverage.	Fast after compilation; predictable.	Higher initial lexicon/rule maintenance.

Recommendation: deploy a hybrid, auditable pipeline. Use regex and DFAs for structured entities, preserve raw text, normalise safe formatting variations, use an FST for known English clinical morphology, and use Porter stemming only as a lower-confidence fallback for out-of-lexicon English tokens. Do not automatically conflate clinically opposite or merely related terms.

The system's likely failure modes are unseen abbreviations, new drug names, misspellings, ambiguous short forms and context-dependent clinical meaning. A clinician may notice a relevant record missing from search or an irrelevant record appearing because two terms were conflated.

Manual effort is concentrated in collecting de-identified records, writing patterns, building and reviewing the lexicon, annotating word forms, and assigning relevance judgements. A statistical/learned method becomes more economical when vocabulary and spelling variation grow faster than experts can maintain rules, provided enough representative labelled data and governance are available.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

A. Impact analysis

The new inputs include OCR character errors (0/O, 1/l, rn/m), line-break hyphenation, lost diacritics, Romanised regional-language words and English/regional code-mixing. The original design assumed clean typed text, English-only morphology, a mostly closed lexicon and exact accept/reject matching. These assumptions no longer hold.

- Character noise causes exact regexes and DFAs to reject otherwise valid IDs, dates and measurements.
- Line-ending hyphens can split a single word into two fragments.
- Out-of-lexicon words reduce FST coverage.
- Romanised regional-language tokens cannot be analysed by English-only morphology.
- Code-mixing makes abbreviation expansion and language-specific stemming context-dependent.

If 20% of incoming tokens are noisy or outside the lexicon, the upper bound on exact-match coverage may fall substantially. The actual precision/recall change must be measured on a labelled noisy sample; it cannot be calculated exactly from the 20% estimate alone.

B. Evaluation of the original design

Synthetic record	Original pipeline behaviour	First failure
OCR: “PT-12O456 fever 3 day s; BP 12O/80 mmHg”	Patient ID and BP patterns reject letter O in numeric positions; “day s” may split duration.	Exact entity regex/DFA validation.
Code-mixed: “patient ku fever irundhuchu, paracetamol 500mg kuduthen”	English tokens and drug strength may be recognised; regional suffixes/verbs remain out of lexicon.	English morphological analyser.
Both: “patlent ku fev er; paracetamol 5O0mg kuduthen”	OCR misspellings and code-mixing combine; entity pattern may miss the drug strength.	Tokenisation/lexical matching.

A small noisy test sample should be annotated and evaluated separately from the clean baseline. Report precision, recall, F1 and analyser coverage for each source type. A weighted automaton or edit-distance matcher can rank likely OCR corrections; a character n-gram language identifier can help detect language at token level. Both are more suitable for uncertain input than adding many exact regex alternatives.

C. Adaptation / redesign

Decision: MODIFY the original finite-state design rather than replace it. Keep the deterministic layer for clean, high-confidence content and add a noise-tolerant front end.

6. OCR correction stage: use weighted edit-distance candidates for common confusions (0/O, 1/l, rn/m), but only auto-correct when confidence is high. Otherwise preserve the original and flag it.
7. Line-break repair: join hyphenated line endings only when the joined token is supported by the lexicon or a language model; retain original offsets.
8. Token-level language identification: label English, regional-language and uncertain tokens using character n-grams or a small trained classifier.
9. Lexicon extension: add reviewed regional-language stems, common Romanised variants and clinical code-mixed phrases.
10. Morphology fallback: use English FST for English tokens, a regional-language analyser when available, and a conservative surface-form index for unknown tokens.
11. Entity extraction: use weighted matching for noisy candidates, followed by strict validation and confidence scores. Keep exact DFA acceptance for high-confidence clean strings.

Expected effect: coverage should improve because likely OCR variants and regional forms can be recovered. Precision may decrease if fuzzy matching is too permissive, so confidence thresholds and human review are necessary. Runtime and annotation effort increase because candidate ranking, language labels and lexicon maintenance are added.

D. Evidence-based defence

The modified design is justified because it preserves the speed, transparency and auditability of finite-state methods for clean text while addressing the two new sources of uncertainty. A weighted edit-distance layer handles character-level OCR noise; token-level language identification and expanded lexicons address code-mixing. The original exact patterns remain valuable as validators after correction, rather than being discarded.

Remaining limitations: finite-state methods do not independently understand clinical intent, resolve all abbreviation senses, interpret negation and temporality across a full note, or infer unseen meanings reliably. OCR correction can also change a clinically important number. Therefore, original text must always be retained, uncertain corrections must be flagged, and high-impact clinical fields should require validation.

Final conclusion

A morphology-aware hospital search layer should combine regular expressions, finite automata, finite-state transducers and conservative normalisation. Inflectional variants can usually be conflated safely, while derivational and semantic relationships require clinical knowledge. For OCR and code-mixed notes, a modified hybrid system with weighted correction and language identification offers better coverage than a strict English-only pipeline, while preserving the original record for safety and audit.

Short glossary

Term	Meaning
DFA	Deterministic finite automaton; exactly one next state for each state/input symbol.
NFA	Non-deterministic finite automaton; may have multiple possible next states and ε transitions.
FST	Finite-state transducer; maps lexical representations to surface forms and vice versa.
Lemma	Dictionary/base form of a word.
Stem	Base-like form produced by a stemmer; may not be a dictionary word.
Precision	Correct retrieved relevant items divided by all retrieved items.
Recall	Correct retrieved relevant items divided by all relevant items.
F1	Harmonic mean of precision and recall.